

CatchUp: An Open-Source, AI-Powered Personal Digest

Zhuoang Tao

Problem Statement

People keep up with topics of interest through websites, blogs, podcasts, and long-form videos. Checking each source separately takes time. The volume and length of this content can make it difficult to identify what deserves closer attention. Users who check less frequently may also miss updates.

This project will develop CatchUp, an open-source application that collects new content from user-selected sources and organizes it into an on-demand digest by topic. CatchUp will help users stay informed and decide which original articles, episodes, or videos to explore in depth.

Target User

CatchUp is designed for people who regularly follow multiple information sources and want to spend less time checking for updates and deciding what to read, watch, or listen to. These users may include students seeking educational resources, developers keeping up with technical blogs, and people following industry news or podcasts. Users choose the topics and sources that interest them. The initial self-hosted version will be aimed at users comfortable following a short setup guide and configuring access to a model API.

Initial Business or Use Case Context

A user runs CatchUp locally or on their own server, configures a supported model API, and adds sources. When ready to catch up, they click Generate Digest. The application collects content published between the previous successful request and the current one, then generates and saves a digest. Users returning after several days can review updates from that entire period in one place.

For example, a student could follow technical blogs and educational podcasts, skim summaries organized by topic, and open the original articles or episodes that interest them. CatchUp reduces manual checking, helps users prioritize content, and gives them control over their sources and model choice. The project will include a public repository, setup documentation, and a deployed demo for course evaluation.



Planned Features

Core functionality

- **Source management:** Add a URL, preview and confirm the identified source, and manage saved sources. Clearly flag unsupported or duplicate sources.
- **On-demand collection:** Find new content within the requested time period. Keep collection records to prevent duplicate entries and distinguish failed checks from successful checks that find no new content.
- **Multiple formats:** Support selected public websites and blogs, plus podcast and video sources with accessible transcripts or captions.
- **AI-generated digests:** Summarize collected content, group it by topic, and retain source names and original links for further reading.
- **Model configuration:** Allow users to provide their own API credentials and choose from supported models. Manage credentials through the backend of their own CatchUp instance.
- **History and feedback:** Save past digests for later review. Add bookmarks, likes, and feedback once the core workflow for collecting content and generating digests is functional.

Stretch features

Future enhancements may include detecting meaningful changes to existing course or event pages and adding a browser extension. The extension could let users add sources directly from their browser or, with their permission, extract content from pages they can access while logged in. Automatic scheduling and push delivery are outside the initial release.

AI Tool Usage Plan

AI tools will support each stage of development, in addition to generating summaries within CatchUp. AI-generated code and other outputs will be reviewed and tested as appropriate.

- **UI/UX and requirements:** Use AI to refine user stories and prototype screens for managing sources, reading digests, and browsing history. Document key changes to requirements and design decisions.
- **Code generation and refactoring:** Use coding assistants to help implement source adapters, model integrations, and application components. Review the resulting code for correctness and maintainability.
- **Automated testing:** Use AI to help develop tests for time range handling, duplicate prevention, source failures, and saving and retrieving digests. Once the core workflow is functional, evaluate summaries against their source material.
- **Code review and quality assurance:** Use AI-assisted code reviews to identify edge cases and weaknesses in error handling and credential management. Manually verify the findings and any resulting fixes.
- **CI/CD and deployment:** Use AI to help configure automated checks, prepare deployment files, and draft setup instructions. Test installation in a clean environment. In the final report, document how AI tools contributed to development, where they fell short, and how those issues were addressed.